

# **System Architecture**

AST-Based Automatic Code Comment Generator

Snigdha Parasa  
24CSB0B73

June 27, 2026

# Contents

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                           | <b>4</b>  |
| 1.1      | Purpose . . . . .                             | 4         |
| 1.2      | Architectural Goals . . . . .                 | 4         |
| <b>2</b> | <b>System Overview</b>                        | <b>4</b>  |
| 2.1      | High-Level Description . . . . .              | 4         |
| 2.2      | Design Philosophy . . . . .                   | 4         |
| <b>3</b> | <b>System Architecture Diagram</b>            | <b>5</b>  |
| 3.1      | Overall System Architecture . . . . .         | 5         |
| 3.2      | Detailed Architecture . . . . .               | 6         |
| <b>4</b> | <b>Module Descriptions</b>                    | <b>6</b>  |
| 4.1      | Module 1: Input/Output Handler . . . . .      | 6         |
| 4.1.1    | Components . . . . .                          | 6         |
| 4.1.2    | Inputs . . . . .                              | 7         |
| 4.1.3    | Outputs . . . . .                             | 7         |
| 4.1.4    | Technologies . . . . .                        | 7         |
| 4.2      | Module 2: Parser Module . . . . .             | 7         |
| 4.2.1    | Components . . . . .                          | 7         |
| 4.2.2    | Inputs . . . . .                              | 8         |
| 4.2.3    | Outputs . . . . .                             | 8         |
| 4.2.4    | Technologies . . . . .                        | 8         |
| 4.2.5    | Key Classes . . . . .                         | 8         |
| 4.3      | Module 3: Analysis Module . . . . .           | 8         |
| 4.3.1    | Components . . . . .                          | 8         |
| 4.3.2    | Inputs . . . . .                              | 9         |
| 4.3.3    | Outputs . . . . .                             | 9         |
| 4.3.4    | Technologies . . . . .                        | 9         |
| 4.3.5    | Key Classes . . . . .                         | 9         |
| 4.4      | Module 4: Comment Generation Module . . . . . | 9         |
| 4.4.1    | Components . . . . .                          | 9         |
| 4.4.2    | Inputs . . . . .                              | 10        |
| 4.4.3    | Outputs . . . . .                             | 10        |
| 4.4.4    | Technologies . . . . .                        | 10        |
| 4.4.5    | Key Classes . . . . .                         | 10        |
| 4.5      | Module 5: Code Integration Module . . . . .   | 10        |
| 4.5.1    | Components . . . . .                          | 10        |
| 4.5.2    | Inputs . . . . .                              | 11        |
| 4.5.3    | Outputs . . . . .                             | 11        |
| 4.5.4    | Technologies . . . . .                        | 11        |
| 4.5.5    | Key Classes . . . . .                         | 11        |
| <b>5</b> | <b>Data Flow Diagram</b>                      | <b>12</b> |
| 5.1      | Data Flow Through the System . . . . .        | 12        |

|           |                                       |           |
|-----------|---------------------------------------|-----------|
| <b>6</b>  | <b>Component Interaction</b>          | <b>13</b> |
| 6.1       | Module Interaction Sequence . . . . . | 13        |
| 6.2       | Interaction Description . . . . .     | 13        |
| <b>7</b>  | <b>Deployment Architecture</b>        | <b>14</b> |
| 7.1       | Physical Deployment . . . . .         | 14        |
| <b>8</b>  | <b>Architectural Patterns</b>         | <b>14</b> |
| 8.1       | Design Patterns Used . . . . .        | 14        |
| 8.1.1     | Visitor Pattern . . . . .             | 14        |
| 8.1.2     | Template Method Pattern . . . . .     | 14        |
| 8.1.3     | Pipeline Pattern . . . . .            | 14        |
| 8.1.4     | Strategy Pattern . . . . .            | 15        |
| <b>9</b>  | <b>Quality Attributes</b>             | <b>15</b> |
| 9.1       | Modularity . . . . .                  | 15        |
| 9.2       | Maintainability . . . . .             | 15        |
| 9.3       | Extensibility . . . . .               | 15        |
| 9.4       | Testability . . . . .                 | 15        |
| <b>10</b> | <b>Conclusion</b>                     | <b>16</b> |

# 1 Introduction

## 1.1 Purpose

This document describes the system architecture for the AST-Based Automatic Code Comment Generator. The architecture follows modular design principles based on compiler design concepts, ensuring clarity, maintainability, and extensibility.

## 1.2 Architectural Goals

- **Modularity:** Clear separation of concerns with well-defined module boundaries
- **Simplicity:** Straightforward design that's easy to understand and implement
- **Explainability:** Transparent processing pipeline with traceable decisions
- **Extensibility:** Easy to add new features, rules, and language support
- **Performance:** Efficient processing with minimal resource usage

# 2 System Overview

## 2.1 High-Level Description

The system follows a pipeline architecture inspired by compiler design. Source code flows through distinct processing stages:

1. **Input Handling:** Read and validate source code files
2. **Parsing:** Generate Abstract Syntax Tree representation
3. **Analysis:** Traverse AST and extract features
4. **Generation:** Create natural language comments
5. **Integration:** Insert comments into source code
6. **Output:** Write commented code to files

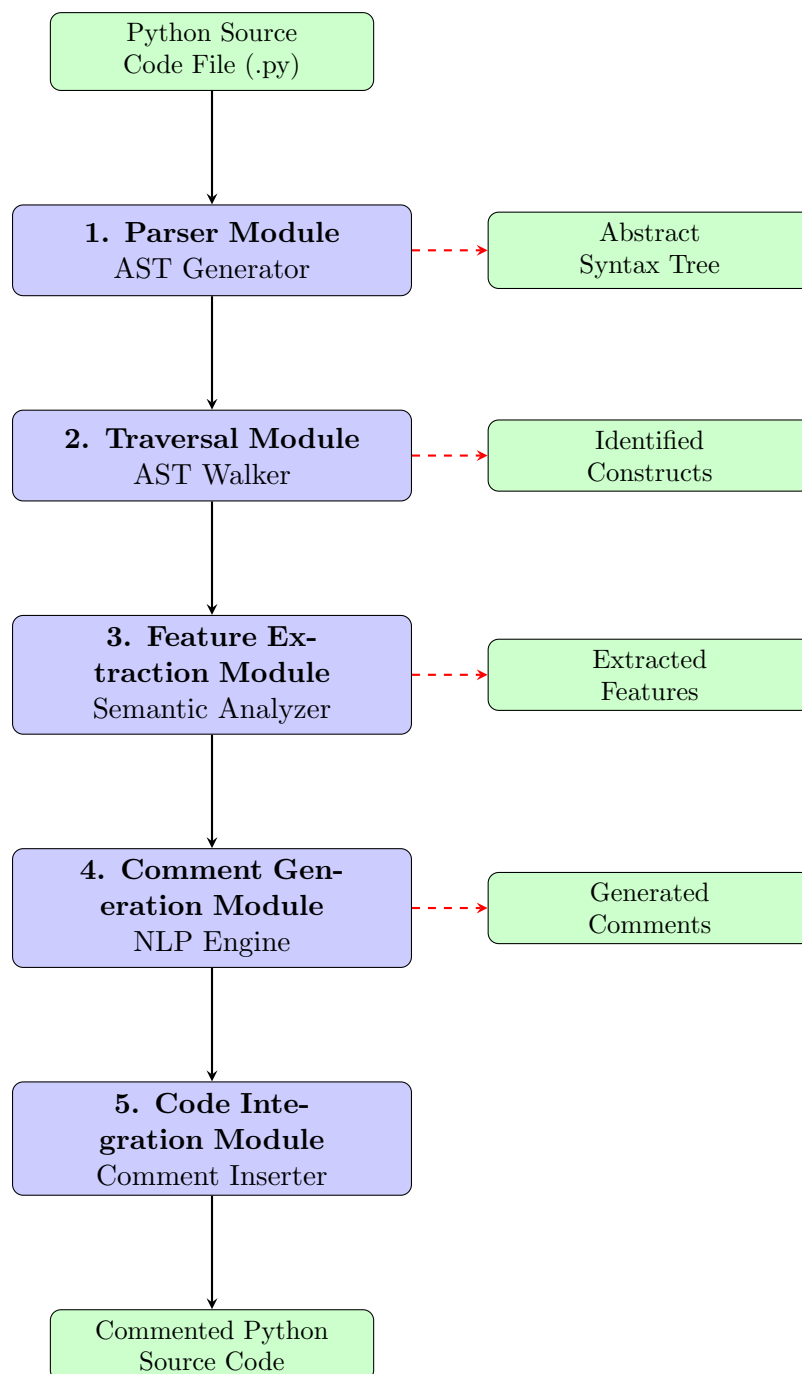
## 2.2 Design Philosophy

The architecture is based on several key principles:

- **Compiler-Inspired:** Leverages traditional compiler phases (lexical, syntactic, semantic)
- **Rule-Based:** Uses deterministic rules rather than probabilistic models
- **Stateless Modules:** Each module operates independently without side effects
- **Data-Driven:** Processing controlled by data flow, not complex control logic

## 3 System Architecture Diagram

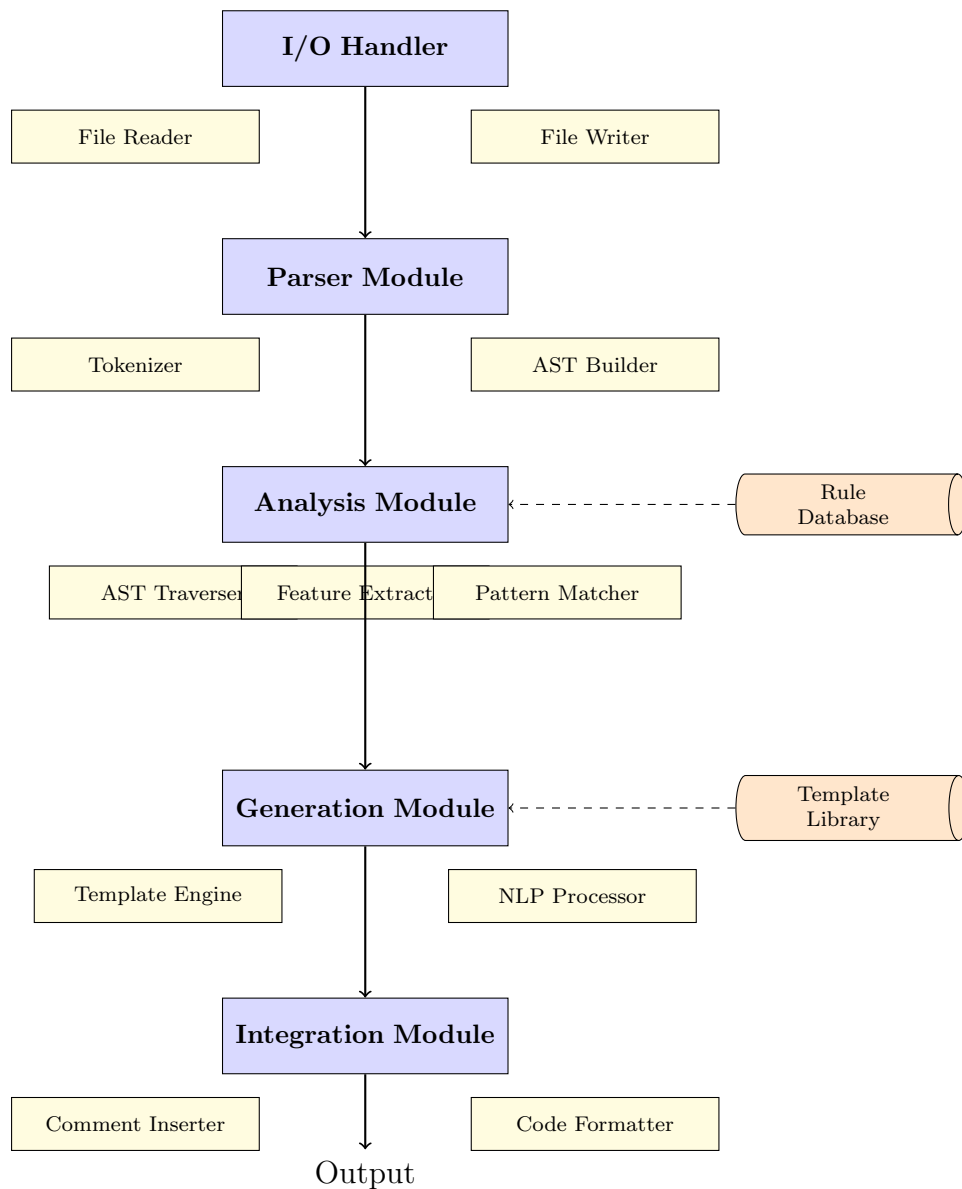
### 3.1 Overall System Architecture



#### Legend:

- **Blue Boxes:** Processing modules
- **Green Boxes:** Data/Artifacts
- **Solid Arrows:** Primary data flow
- **Dashed Arrows:** Intermediate outputs

### 3.2 Detailed Architecture



## 4 Module Descriptions

### 4.1 Module 1: Input/Output Handler

**Responsibility:** Manage file system interactions

#### 4.1.1 Components

##### 1.1 File Reader

- Read Python source files from file system
- Validate file existence and permissions
- Handle file encoding (UTF-8)

- Support batch file processing

### **1.2 File Writer**

- Write commented code to output files
- Preserve file permissions and metadata
- Create backup of original files
- Handle output directory creation

#### **4.1.2 Inputs**

- File paths or directory paths
- Configuration parameters

#### **4.1.3 Outputs**

- Source code strings
- File metadata

#### **4.1.4 Technologies**

- Python `os` module for path handling
- Python `pathlib` for file operations
- Built-in `open()` for file I/O

## **4.2 Module 2: Parser Module**

**Responsibility:** Convert source code into Abstract Syntax Tree

### **4.2.1 Components**

#### **2.1 Tokenizer (Built-in)**

- Break source code into tokens (handled by Python's parser)
- Identify keywords, operators, identifiers, literals

#### **2.2 AST Builder**

- Parse tokens into AST structure
- Validate syntax correctness
- Create node objects for each construct
- Build hierarchical tree structure

#### 4.2.2 Inputs

- Python source code string

#### 4.2.3 Outputs

- AST object (Python `ast.Module`)
- Syntax error information (if parsing fails)

#### 4.2.4 Technologies

- Python `ast` module - `ast.parse()`
- Handles lexical and syntactic analysis

#### 4.2.5 Key Classes

- `CodeParser` - Main parser interface
- Uses built-in AST nodes: `FunctionDef`, `ClassDef`, `If`, `For`, etc.

### 4.3 Module 3: Analysis Module

**Responsibility:** Traverse AST and extract meaningful information

#### 4.3.1 Components

##### 3.1 AST Traverser

- Implement visitor pattern (`ast.NodeVisitor`)
- Visit each node in AST systematically
- Maintain context during traversal
- Handle nested structures

##### 3.2 Feature Extractor

- Extract structural features (names, parameters, types)
- Analyze naming patterns
- Compute complexity metrics
- Identify code smells

##### 3.3 Pattern Matcher

- Recognize common coding patterns
- Identify design patterns (factory, singleton, etc.)
- Detect security-relevant patterns
- Match function naming conventions



#### 4.3.2 Inputs

- AST object
- Analysis rules and patterns

#### 4.3.3 Outputs

- Feature dictionaries for each construct
- Pattern match results
- Metadata annotations

#### 4.3.4 Technologies

- Python `ast.NodeVisitor` - Traversal pattern
- Regular expressions for pattern matching
- Custom rule definitions

#### 4.3.5 Key Classes

- `ASTTraverser` - Implements visitor pattern
- `FeatureExtractor` - Extracts features from nodes
- `PatternMatcher` - Identifies code patterns
- `SecurityAnalyzer` - Detects security patterns

### 4.4 Module 4: Comment Generation Module

**Responsibility:** Generate natural language comments from extracted features

#### 4.4.1 Components

##### 4.1 Template Engine

- Manage comment templates
- Fill templates with extracted data
- Support multiple template styles
- Handle template variations

##### 4.2 NLP Processor

- Convert identifiers to natural language
- Generate grammatically correct sentences
- Apply naming convention rules
- Ensure comment quality

#### 4.4.2 Inputs

- Feature dictionaries
- Comment templates
- Generation rules

#### 4.4.3 Outputs

- Generated comment text
- Comment type (docstring/inline/block)
- Insertion position information

#### 4.4.4 Technologies

- String template system
- Regular expressions for text processing
- Rule-based NLP techniques

#### 4.4.5 Key Classes

- `CommentGenerator` - Main generation interface
- `TemplateManager` - Template selection and management
- `NLPEngine` - Natural language processing
- `QualityChecker` - Validate generated comments

### 4.5 Module 5: Code Integration Module

**Responsibility:** Insert generated comments into source code

#### 4.5.1 Components

##### 5.1 Comment Inserter

- Insert docstrings into functions/classes
- Add inline comments at appropriate locations
- Handle indentation correctly
- Preserve existing code structure

##### 5.2 Code Formatter

- Ensure proper Python syntax
- Format comments according to PEP 257
- Maintain code style consistency
- Validate output correctness

#### 4.5.2 Inputs

- Original source code
- Generated comments with positions
- AST with location information

#### 4.5.3 Outputs

- Commented source code
- Validation report

#### 4.5.4 Technologies

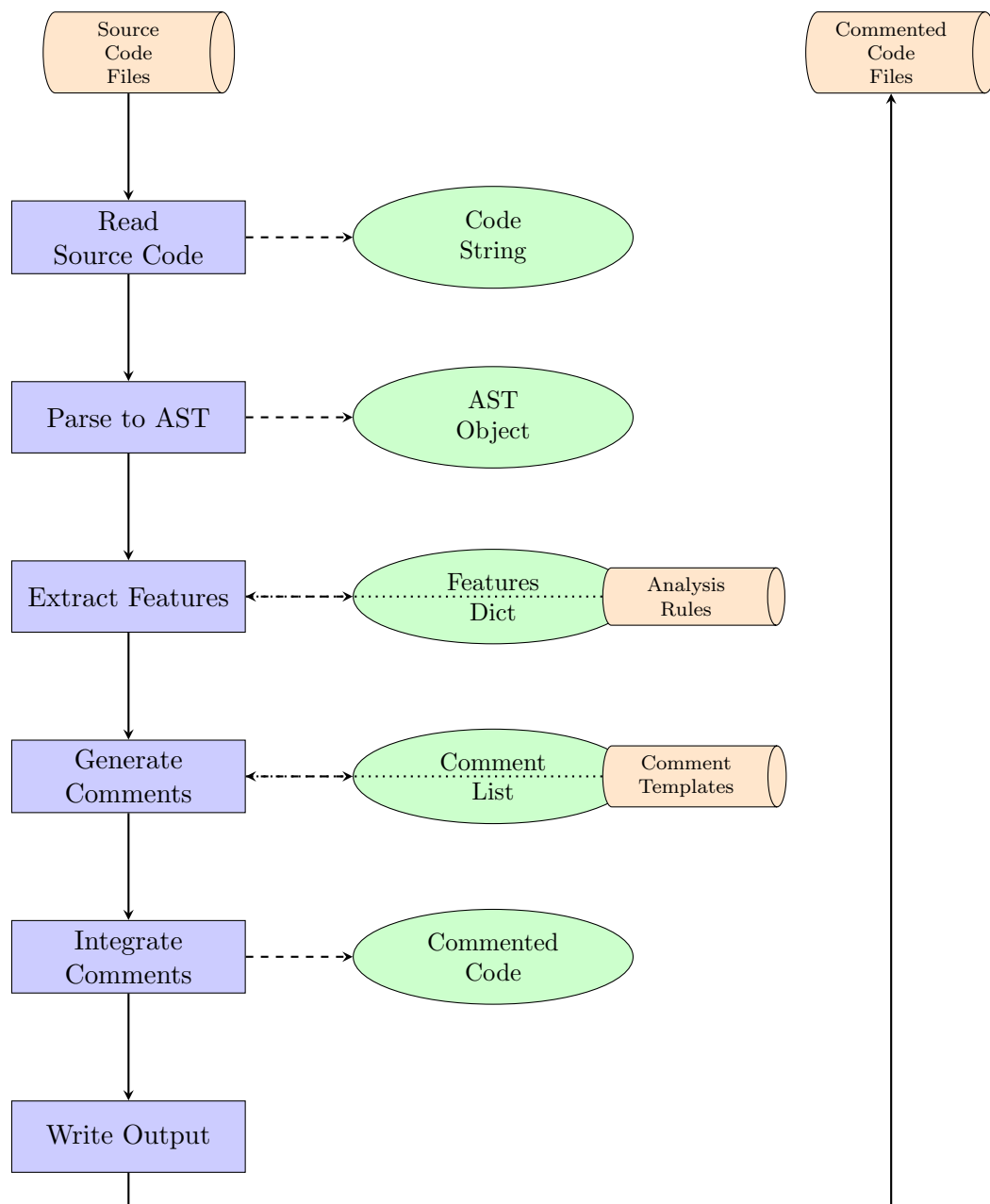
- Python `astor` - AST to source conversion
- String manipulation for insertion
- Syntax validation

#### 4.5.5 Key Classes

- `CodeIntegrator` - Main integration interface
- `DocstringInserter` - Add function/class docs
- `InlineCommentInserter` - Add inline comments
- `SyntaxValidator` - Verify output correctness

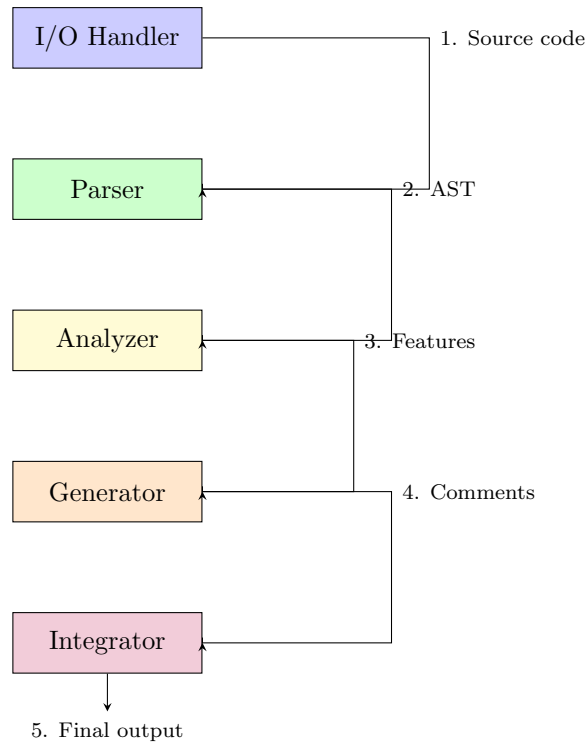
## 5 Data Flow Diagram

### 5.1 Data Flow Through the System



## 6 Component Interaction

### 6.1 Module Interaction Sequence



### 6.2 Interaction Description

1. **I/O → Parser:** Source code string passed to parser
2. **Parser → Analyzer:** AST object passed to analyzer
3. **Analyzer → Generator:** Feature dictionaries passed to generator
4. **Generator → Integrator:** Generated comments with positions passed to integrator
5. **Integrator → Output:** Final commented code written to file

## 7 Deployment Architecture

### 7.1 Physical Deployment

#### Deployment Characteristics:

- **Platform:** Standalone desktop application
- **Dependencies:** Python 3.8+, standard library
- **Installation:** Simple file copy, no compilation
- **Execution:** Command-line interface
- **No Network:** Entirely local processing

## 8 Architectural Patterns

### 8.1 Design Patterns Used

#### 8.1.1 Visitor Pattern

**Usage:** AST traversal

**Benefit:** Cleanly separates tree structure from operations performed on nodes

**Implementation:** Python's `ast.NodeVisitor`

#### 8.1.2 Template Method Pattern

**Usage:** Comment generation

**Benefit:** Defines skeleton of comment generation while allowing customization

**Implementation:** Template-based comment system

#### 8.1.3 Pipeline Pattern

**Usage:** Overall system flow

**Benefit:** Clear separation of processing stages

**Implementation:** Sequential module chain

#### 8.1.4 Strategy Pattern

**Usage:** Different comment generation strategies

**Benefit:** Easily swap between different generation approaches

**Implementation:** Pluggable comment generators

## 9 Quality Attributes

### 9.1 Modularity

- Each module has single, well-defined responsibility
- Modules communicate through clear interfaces
- Low coupling between modules
- High cohesion within modules

### 9.2 Maintainability

- Clear module boundaries
- Comprehensive documentation
- Consistent coding standards
- Easy to locate and fix bugs

### 9.3 Extensibility

- New rules easily added to analyzer
- New templates easily added to generator
- Architecture supports future language support
- Pluggable components

### 9.4 Testability

- Each module independently testable
- Clear inputs and outputs
- No hidden dependencies
- Mockable components

## 10 Conclusion

The system architecture provides:

- **Clear Structure:** Well-defined modules with specific responsibilities
- **Compiler Principles:** Based on traditional compilation phases
- **Transparency:** Each processing step is explicit and traceable
- **Flexibility:** Easy to extend and modify
- **Simplicity:** Straightforward implementation path

This architecture ensures the system can be successfully implemented within the academic timeline while delivering reliable, maintainable, and explainable code comment generation.